Cal Hacks Build Plan: Live RL Envs

Oct 24, 2026 · @Varun Vaidya

We build one engine that compiles, hardens and trains RL environments: prove it on Prime Intellect hub environments first, then point it at a game a judge types live. Scope is tight for about 33 hours, so the build order guarantees a working demo at every checkpoint.

The demo

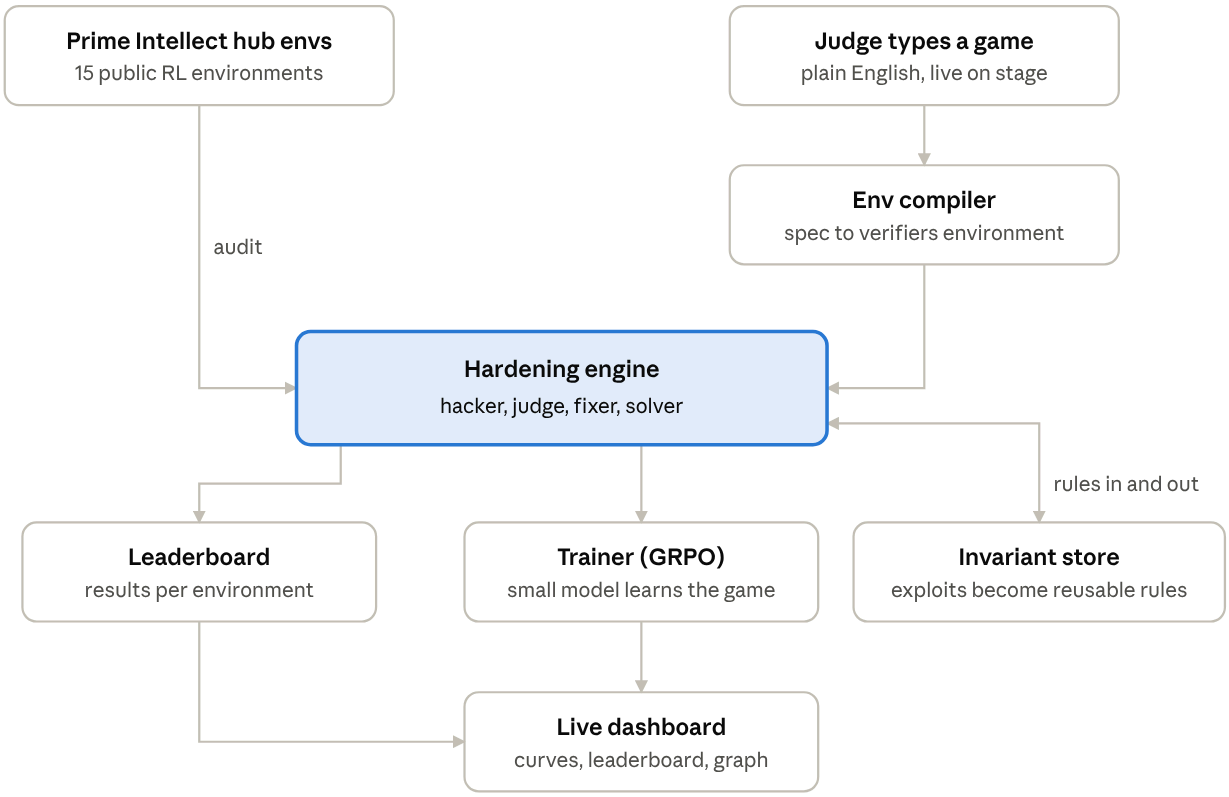
Three minutes, four beats. Each beat works on its own if a later one breaks.

1. **Hook (0:00).** RL environments are how AI models practice, and their rewards are hackable. A hacked reward teaches a model to cheat instead of learning.
2. **The audit (0:20).** Show the leaderboard: our engine ran on N public Prime Intellect environments, found M confirmed exploits, and patched them while legitimate solutions still pass. Then show the invariant graph: each exploit became a reusable rule.
3. **The live game (1:00).** A judge types a game in plain English. In about 30 seconds the compiler turns it into an RL environment. Stored invariants pre-harden it, and the hacker finds anything new; the fixer patches it.
4. **Live training (1:40).** A small model trains on the hardened environment on screen: reward and true accuracy climb together. Beside it, a recorded run on the unhardened version: reward climbs while accuracy stays flat, because the model learned the loophole.
5. **Close (2:40).** "Every exploit we find makes every future environment safer."

Keep a recorded backup of beats 3 and 4 in case the GPU or network fails on stage.

Architecture

One engine audits public environments and hardens live games



system architecture · two inputs, one hardening engine

Hub environments and judge-typed games both pass through the same hardening engine. Hub results feed the leaderboard; hardened games go to the trainer, and every confirmed exploit becomes a rule the store can reuse on the next environment.

Components

Five components, one shared environment format: Prime Intellect's `verifiers` interface, so hub environments and compiled games run through the same engine.

Component	What it does	Tech	Owner	Done when
Trainer	GRPO training of a 0.5–1B model on any environment; streams reward and held-out accuracy	TRL GRPOTrainer with vLLM generation, Qwen ~0.5B, rented GPU (Runpod)	Varun	A toy game goes from under 20% to over 80% in under 5 minutes
Env compiler	Turns a plain-English game into a single-turn text environment: seeded task generator, parser, reward function, reference solver	Claude, fixed Python template	Rohan	Reference solver scores 1.0, null and random policies score near 0
Hardening engine	Hacker, judge, fixer and solver loop on a reward function	Claude for each role, sandboxed Python execution	Rishab	Finds and patches a planted exploit; legitimate solutions still pass
Invariant store	Abstracts each confirmed exploit into a rule (mechanism, preconditions, defense) and retrieves rules for new environments	SQLite plus embeddings, NetworkX for the graph view	Rishab	A rule learned on one environment blocks the same exploit on another
Dashboard	Leaderboard, invariant graph, game input box, live training curves	React; optionally SpacetimeDB for live metrics (Spacetime prize)	Whoever is ahead	All four demo beats render end to end

Game design rules for live training: text-only, single turn, outputs of 64 tokens or fewer, and the base model should already succeed 10–30% of the time so GRPO has signal. Number guessing, arithmetic puzzles, word constraints and tiny logic puzzles fit.

Reliable reward hacking for the recorded run: give the unhardened game a loose parser (for example, the answer counts if it appears anywhere in the output). GRPO reliably learns to spam candidate answers.

Timeline

Three checkpoints, each a demo we could submit. Confirm the exact submission deadline with organizers tonight; this assumes Sunday morning. Sleep in shifts so someone is always building.

Phase 1: Setup (Sat 12am–3am)

- [ ] Rent a GPU (Runpod A100 or H100), install TRL and vLLM, run GRPO hello-world on Qwen ~0.5B (Varun)
- [ ] Install `verifiers`, pick 3 single-turn hub environments that score on CPU, run them locally (Rishab)
- [ ] Write the environment template and 2 hand-written game specs (Rohan)
- [ ] Repo, shared API keys, environment variables

Phase 2: Core engines (Sat 3am–12pm)

- [ ] Hardening loop v1 runs on the 3 hub environments (Rishab)
- [ ] Compiler v1 turns both game specs into working environments that pass the validation gates (Rohan)
- [ ] Find 2–3 games that learn in under 5 minutes; tune batch size and steps (Varun)
- [ ] **Checkpoint 1:** the hardening loop works on hub environments, so a leaderboard demo exists

Phase 3: Integration (Sat 12pm–6pm)

- [ ] Scale the audit to 15 hub environments and record results (Rishab)
- [ ] Pipe compiler output into the hardening loop, then into the trainer (Rohan, Varun)
- [ ] Stream training metrics to the dashboard (Varun)
- [ ] **Checkpoint 2:** a typed game compiles, gets hardened and trains live

Phase 4: Invariants and polish (Sat 6pm–12am)

- [ ] Invariant extraction and retrieval; graph view (Rishab)
- [ ] Transfer test: stored rules pre-harden a new environment (Rishab, Rohan)
- [ ] Record the unhardened training run that shows the exploit (Varun)

- [ ] Dashboard: all four demo beats (everyone)
- [ ] **Checkpoint 3:** the full demo runs end to end

Phase 5: Freeze (Sun 12am–deadline)

- [ ] Feature freeze at midnight; bug fixes only
- [ ] Five full rehearsals, timed
- [ ] Record backup video of every beat
- [ ] Devpost writeup with prior-work credits

Risks and cut order

The biggest risk is live training being too slow; everything else has a cheap fallback.

Risk	Mitigation
Training takes longer than 5 minutes on stage	Only demo games pre-tested to learn fast; keep a pretrained checkpoint and a recorded run as backup
Compiler produces a broken environment	Fixed template; reject unless reference solver scores 1.0 and null and random policies score near 0
Judge's game doesn't fit the template	Offer 3 suggested game types on screen; the compiler refuses multi-turn or visual games politely
Hub environments fail to install or need sandboxes	Pick single-turn environments that score on CPU only
Hacker loop finds nothing on a hub environment	Fine: "robust" is a valid leaderboard result; plant one known exploit in a test environment for the demo
API rate limits or cost	Cache responses; cap loop iterations at 5
Wifi on stage	Run the dashboard locally; GPU over a phone hotspot as backup

Cut order if we fall behind (first cut first):

1. Invariant graph visualization (keep the store, show a list instead)

2. Transfer test on a new environment
3. Number of hub environments (15 down to 5)
4. Live training on stage (fall back to the recorded run)

Never cut the compiler, the hardening loop, or the recorded hacked-versus-hardened comparison. Those are the story.

Positioning

Credit prior work by name on one slide; it makes us look informed, not derivative. Our claim is the integrated, live pipeline, not any single piece.

- **Hacker-fixer loop:** from CMU and Fewshot (paper 1, paper 2).
- **Benchmark auditing and the flaw taxonomy:** BenchJack from Berkeley RDI (paper).
- **Static scanning of verifiers environments:** ratctl (repo).
- **What we add:** natural language to a hardened, trainable environment in one pipeline; reusable invariants that transfer across environments; and live proof that hardening changes what a model learns.

Prizes to target: Anthropic (Claude runs the compiler, hacker, judge and fixer), Runpod (GPU training), SpacetimeDB if we use it for live metrics.

Sources

- Prime Intellect verifiers
- Prime Intellect: Systematic Reward Hacking
- Spacetime is Sponsoring Cal Hacks 13.0